



COMSATS University Islamabad
Sahiwal Campus
(Department of Computer Sciences)

Course Title:	OOP				Course Code:	CSC—241	Credit Hours:	4(3+1)
Course Instructor:	Mr. Shehzad Ali				Program Name:	BSSE		
Lab Instructor:	Mr. Shehzad Ali							
Semester:	3rd	Batch:	FA24	Section:	B	Submission Date:	20-10-2025	
Time Allowed:	50 minutes				Maximum Marks:		10	
Student's Name:					Reg. No.			

Assignment No. 2

CLO-4 , GA2-4

Q1: Apply the concept of encapsulation to demonstrate an understanding of secure data handling and solve real-world problems using Object-Oriented Programming. (10 Marks)

You are a software engineer at a bank, and your team is responsible for developing a **Bank Management System** that securely handles customer account details and transactions. To ensure proper organization, secure data handling, and code reusability, your manager asks you to design the system using **Object-Oriented Programming (OOP)** principles.

Part 1 – Encapsulation

Create a class named **BankAccount** that encapsulates essential details such as the **account holder's name**, **account number**, and **balance**.

All attributes must be declared as **private**, and you should provide **public setter and getter methods** to safely access and modify them.

Include methods **deposit()** and **withdraw()** to perform transactions securely.

Part 2 – Composition (Has-A Relationship)

Introduce another class named **Customer**, which **has-a** relationship with **BankAccount**.

Each **Customer** object should have private attributes such as **customerID**, **contactNumber**, and a **BankAccount** object to represent the account owned by that customer.

This demonstrates **Composition**, where a **Customer** *has a* **BankAccount**.

Provide methods in the **Customer** class to display complete customer details along with account information.

Part 3 – Inheritance

To handle different account types, create subclasses **SavingsAccount** and **CurrentAccount** that **inherit** from the **BankAccount** class.

Each subclass should override or extend behavior (for example, `SavingsAccount` may have interest calculation, and `CurrentAccount` may have a minimum balance requirement).

Part 4 – Comparison and Display

In the `main()` method:

- Create two `Customer` objects and take input from the user.
- Compare their accounts to check whether they belong to the same account holder or not.
- Compare the account balances and display the customer with the **higher balance**.